



中山大學

SUN YAT-SEN UNIVERSITY

移动机器人规划与控制期末作业报告

王相轩 23354154

2026 年 1 月 7 日

目录

1	题目1：坐标系转换	2
1.1	已知条件	2
1.2	目标	2
1.3	计算步骤	2
1.4	结果分析	4
2	题目2：开放题解答	5
2.1	A*轨迹规划的启发式与路径简化	5
2.1.1	启发式函数与tie_breaker的影响分析	5
2.1.2	路径简化参数的影响分析	6
2.2	SO(3)位置控制器的力姿态生成	6
2.2.1	控制器各项作用分析	6
2.2.2	重载机体参数调整策略	6
2.3	动力学建模与约束	7
2.3.1	建模简化对控制分配的影响分析	7
2.3.2	气动阻力补偿策略	8
3	实验任务1：动力学模型补全	9
3.1	平移动力学模型补全	9
3.2	旋转动力学模型补全	10
4	实验任务2：无人机的前端规划模块补全	10
4.1	算法原理概述	10
4.2	补全思路详解	11
4.2.1	启发式函数设计（STEP 1.1）	11
4.2.2	主循环逻辑（STEP 1.2）	11
4.2.3	路径回溯（STEP 1.3）	12
5	实验任务3：控制器参数调试	13
5.1	调参思路与过程分析	13
5.1.1	参数影响理论分析	13
5.1.2	详细调参过程记录	13
5.2	调参效果综合分析	15
6	附录	16
6.1	最后一次提交前的完整git commit记录	16

1 题目1: 坐标系转换

1.1 已知条件

- 无人机在世界坐标系 $\{W\}$ 下的姿态（由 `tracking.csv` 提供）以四元数 ${}^W_B\mathbf{q} = [q_w, q_x, q_y, q_z]^T$ 表示。
- 末端执行器固连坐标系 $\{D\}$ 相对于无人机机体坐标系 $\{B\}$ 的姿态变化由旋转矩阵 B_R_D 描述，其矩阵元由公式(1)给出：

$${}^B_R_D(t) = \begin{bmatrix} \cos \omega t & -\sin \omega t \cos \alpha & \sin \omega t \sin \alpha \\ \sin \omega t & \cos \omega t \cos \alpha & -\cos \omega t \sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}$$

其中， $\omega = 0.5 \text{ rad/s}$, $\alpha = \pi/12$ 。

1.2 目标

计算末端执行器在世界坐标系 $\{W\}$ 下的姿态，同样以四元数 ${}^W_D\mathbf{q}$ 表示，并绘制四元数各分量随时间的变化曲线。

1.3 计算步骤

计算过程的核心是坐标系的链式变换。世界系到末端执行器系的旋转矩阵可以通过机体系进行传递，整体流程如下：

步骤1: 辅助函数定义

- 思路解释：** 为了简化后续计算流程，定义三个核心辅助函数：
1. 四元数归一化：四元数作为姿态表示需满足模长为1，且统一实部（ q_w ）非负，保证姿态表示的唯一性和时间序列连续性；
 2. 四元数转旋转矩阵：将无人机的四元数姿态转换为旋转矩阵，便于进行坐标系变换的矩阵乘法；
 3. 旋转矩阵乘法：封装矩阵点乘操作，实现旋转矩阵的链式变换。

```

1 # Function to normalize : 四元数归一化函数quaternion
2 def normalize_quaternion(q):
3     norm = np.linalg.norm(q) # 计算四元数的范数（模长）L2
4     if norm == 0: # 避免除零错误
5         return q
6     q_normalized = q / norm # 归一化到单位模长
7     if q_normalized[0] < 0: # 保证实部非负，避免姿态表示的符号歧义q_w
8         q_normalized = -q_normalized
9     return q_normalized
10
11 # Function to convert quaternion to rotation : 四元数转旋转矩阵matrix
12 def quaternion_to_rotation_matrix(q):
13     # 的scipyRotation.接收from_quat[x,y,z,w顺序的四元数，需匹配输入格式]
14     return R.from_quat([q[0], q[1], q[2], q[3]]).as_matrix()
15
16 # Function to multiply two rotation : 旋转矩阵乘法（链式变换）matrices
17 def multiply_rotation_matrices(R1, R2):
18     # 旋转矩阵乘法对应坐标系的连续变换，使用点乘实现矩阵乘法numpy

```

```
19 return np.dot(R1, R2)
```

步骤2: 计算世界系到机体系的旋转矩阵

思路解释: CSV文件中读取的无人机四元数 ${}^W_B \mathbf{q}$ 表示“机体系到世界系”的旋转（即 ${}^W \mathbf{R}_B$ 的逆），因此需先将四元数转换为“机体系到世界系”的旋转矩阵，再通过转置（正交矩阵的逆等于转置）得到“世界系到机体系”的旋转矩阵 ${}^W \mathbf{R}_B$ 。需特别注意scipy库的四元数输入顺序（ $[q_x, q_y, q_z, q_w]$ ）与题目定义（ $[q_w, q_x, q_y, q_z]$ ）的差异，确保数据顺序正确。

$${}^W \mathbf{R}_B = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}$$

代码实现片段:

```
1 # Drone's rotation : 无人机机体系到世界系的旋转矩阵matrix
2 # 从行数据中提取无人机四元数, 顺序为CSV[qx, qy, qz, qw (匹配]输入要求) scipy
3 drone_quat = np.array([qx, qy, qz, qw])
4 drone_rot_mat = quaternion_to_rotation_matrix(drone_quat) # 转换为旋转矩阵
5
6 # World to Drone rotation : 世界系到无人机机体系的旋转矩阵matrix
7 # 正交旋转矩阵的逆等于其转置, 因此转置得到世界系到机体系的变换
8 R_W_B = drone_rot_mat.T # 无人机旋转矩阵的转置 (逆)
```

步骤3: 计算机体系到末端执行器系的旋转矩阵

思路解释: 根据题目给出的公式(1)，代入当前时间 t 计算 ${}^B \mathbf{R}_D$ 。其中 $\theta = \omega \cdot t$ 是随时间变化的旋转角， α 是固定倾斜角，需严格按照矩阵元素的位置填充三角函数计算结果，确保旋转矩阵的正确性（符合坐标系旋转变换的定义）。

```
1 # End-effector's rotation relative to : 机体系到末端执行器系的旋转矩阵drone
2 theta = omega * t # 随时间变化的旋转角, 为角速度omega
3 R_B_D = np.array([
4     [np.cos(theta), -np.sin(theta) * np.cos(alpha), np.sin(theta) * np.sin(alpha)],
5     [np.sin(theta), np.cos(theta) * np.cos(alpha), -np.cos(theta) * np.sin(alpha)],
6     [0, np.sin(alpha), np.cos(alpha)]
7 ]) # 严格按照题目公式构建旋转矩阵(1)
```

步骤4: 计算世界系到末端执行器系的旋转矩阵

思路解释: 坐标系变换满足链式法则，世界系到末端执行器系的旋转矩阵等于“世界系到机体系”的旋转矩阵与“机体系到末端执行器系”的旋转矩阵的乘积。矩阵乘法的顺序需严格遵循“从左到右”的坐标系变换逻辑，确保变换关系正确。

$${}^W \mathbf{R}_D(t_i) = {}^W \mathbf{R}_B(t_i) \cdot {}^B \mathbf{R}_D(t_i)$$

代码实现:

```
1 # World to End-effector rotation : 世界系到末端执行器系的旋转矩阵matrix
2 # 矩阵乘法实现坐标系链式变换, (世界→机体) R_W_B * (机体→末端) R_B_D = (世界→末端) R_W_D
3 R_W_D = multiply_rotation_matrices(R_W_B, R_B_D)
```

步骤5: 旋转矩阵转四元数并后处理

思路解释: 最终需要输出末端执行器在世界系下的四元数姿态, 因此需将旋转矩阵转回四元数。需注意两个关键细节: 1. scipy库的‘as_quat()’返回四元数顺序为 $[x, y, z, w]$, 需调整为题目要求的 $[w, x, y, z]$; 2. 对转换后的四元数进行归一化和符号修正, 保证实部非负, 确保姿态表示的唯一性和时间序列的连续性。

1. 旋转矩阵转四元数: scipy返回 $[x, y, z, w]$ 顺序, 需调整为 $[w, x, y, z]$
2. 归一化: 保证四元数模长为1
3. 符号修正: 确保 $q_w \geq 0$, 保证时间序列连续性

```
1 # Convert rotation matrix to : 旋转矩阵转四元数并格式修正quaternion
2 quat_W_D = R.from_matrix(R_W_D).as_quat() # 返回scipy [x, y, z, w] 顺序的四元数
3 quat_W_D = np.array([quat_W_D[3], quat_W_D[0], quat_W_D[1], quat_W_D[2]]) # 调整为 [w, x, y, z] 顺序
4 quat_W_D_normalized = normalize_quaternion(quat_W_D) # 归一化符号修正, 保证+≥q_w0
```

1.4 结果分析

通过上述步骤, 成功计算出每个时间点末端执行器在世界坐标系下的归一化四元数姿态 ${}^W\mathbf{q}(t_i)$, 绘制的变化曲线如下:

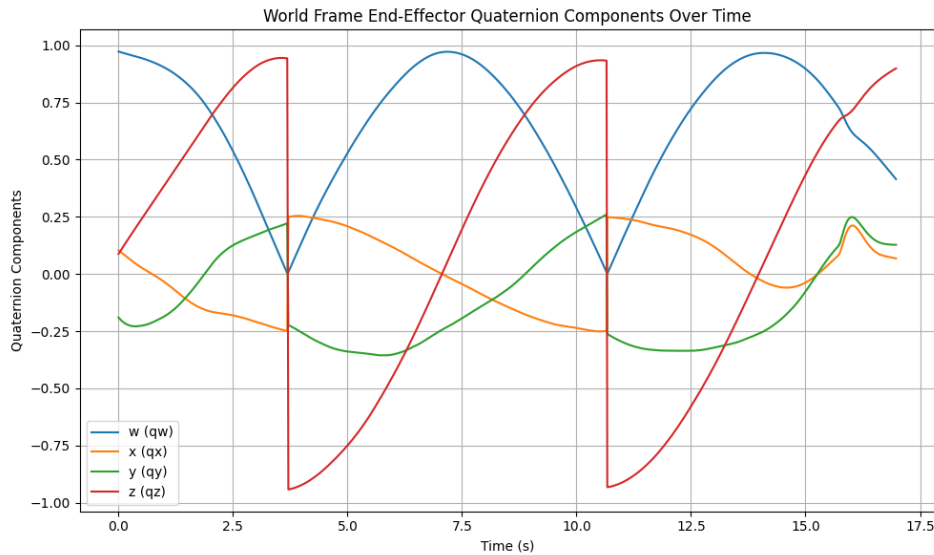


图 1: 末端执行器在世界系下的四元数分量随时间变化曲线

2 题目2: 开放题解答

2.1 A*轨迹规划的启发式与路径简化

2.1.1 启发式函数与tie_breaker的影响分析

现有tie_breaker的作用机制:

在A*算法中, 启发式函数 $h(n)$ 用于估计从节点 n 到目标节点的代价。tie_breaker通过轻微放大启发式值来影响开集节点的扩展顺序:

$$\begin{aligned} f_{\text{new}}(n) &= f(n) \times (1 + \epsilon) \\ &= [g(n) + h(n)] \times (1 + \epsilon) \end{aligned}$$

其中 ϵ 为很小的正数 (如0.001)。

关于其对开集拓展顺序的影响: 首先可以打破平局, 当多个节点具有相同的 f 值时, tie_breaker使得距离起点更近 (g 值更小) 的节点具有更小的调整后 f 值; 其次可以减少”走之”路径: 通过优先扩展靠近起点的节点, 算法更倾向于生成直线路径, 减少不必要的方向变化。

其次, 其对路径平滑性的影响包括: 减少路径中的锯齿状波动, 提高路径的平滑度; 但在获得更实用的飞行路径的同时可能牺牲理论上的最短路径最优性。

偏好平面飞行的修改方案:

为减少不必要的高度变化, 可修改启发式函数:

```
1 double getHeu(MappingNodePtr node1, MappingNodePtr node2) {
2     double dx = abs(node1->index(0) - node2->index(0));
3     double dy = abs(node1->index(1) - node2->index(1));
4     double dz = abs(node1->index(2) - node2->index(2));
5
6     // 增加高度变化的惩罚权重
7     double height_penalty = 2.0;
8     double h = dx + dy + height_penalty * dz;
9
10    // tie_breaker
11    double tie_breaker = 1.0 + 1.0/1000.0;
12    return h * tie_breaker;
13 }
```

或修改边权计算:

```
1 // 在函数中AstarGetSucc
2 edgeCostSets.push_back(sqrt(dx*dx + dy*dy + height_penalty*dz*dz));
```

对可行性和最优性的影响分析: 可行性方面, 修改后仍能保证找到可行路径, 因为障碍物检测逻辑不变; 最优性方面, 牺牲几何最短路径最优性, 但获得更符合无人机动力学特性的实用路径; 计算效率方面, 可能增

加扩展节点数量，但改善了路径质量。

2.1.2 路径简化参数的影响分析

path_resolution过大的影响：跟踪误差方面，由下面公式，路径点过少，相邻点间距过大，无人机需要大幅调整姿态：

$$\text{最大跟踪误差} \approx \frac{\text{路径段长度}^2}{8 \times R_{\min}}$$

其中 $R_{\min}$ 为无人机最小转弯半径。

其次在安全性方面，简化后的路径可能过于靠近障碍物，缺乏安全裕度。

path_resolution过小的影响：路径点过多，增加控制器计算复杂度；包含过多细节，导致频繁的姿态调整，且增加了路径数据存储空间。

2.2 SO(3)位置控制器的力姿态生成

2.2.1 控制器各项作用分析

阅读代码可知，控制器力计算公式为：

$$\begin{aligned} \mathbf{F} = & \mathbf{K}_x \circ (\mathbf{p}_d - \mathbf{p}) + \mathbf{K}_v \circ (\mathbf{v}_d - \mathbf{v}) \\ & + m\mathbf{a}_d + m\mathbf{K}_a \circ (\mathbf{a}_d - \mathbf{a}) + mg\mathbf{e}_z \end{aligned}$$

其各项物理意义为：

项	作用描述
$\mathbf{K}_x \circ (\mathbf{p}_d - \mathbf{p})$	位置误差比例控制，提供向目标收敛的基本力
$\mathbf{K}_v \circ (\mathbf{v}_d - \mathbf{v})$	速度误差微分控制，抑制超调，提高稳定性
$m\mathbf{a}_d$	期望加速度前馈，提前补偿动态变化
$m\mathbf{K}_a \circ (\mathbf{a}_d - \mathbf{a})$	加速度误差补偿，改善跟踪性能
$mg\mathbf{e}_z$	重力补偿，维持高度稳定

由上述公式，在大误差或传感器噪声情况下对ka项截断或限幅的必要性包括：

- 大误差情况下，当 $|\mathbf{p}_d - \mathbf{p}| > 3$ 时，截断ka可以防止控制力饱和：

$$\mathbf{K}_a = [\min(|e_x|, 3) \times 0.2, \min(|e_y|, 3) \times 0.2, \min(|e_z|, 3) \times 0.2]$$

- 考虑传感器噪声：加速度传感器噪声 η_a 会被放大，因此需要对Ka做限幅：

$$\Delta F_{\text{noise}} = m\mathbf{K}_a \circ \eta_a$$

2.2.2 重载机体参数调整策略

参数调整方案：

```
1 // 质量更新
2 setMass(new_mass);
3
4 // 增益调整 (基于质量比例)
5 double mass_ratio = new_mass / original_mass;
6 Eigen::Vector3d new_kx = kx * sqrt(mass_ratio);
7 Eigen::Vector3d new_kv = kv * sqrt(mass_ratio);
```

调整原理分析：根据牛顿第二定律 $F = ma$ ，推力与质量成正比，当机体更重时，为了保持相似的加速度跟踪，采用平方根缩放保持相似的阻尼比：

$$\zeta = \frac{k_v}{2\sqrt{mk_x}} \Rightarrow \text{保持}\zeta\text{不变需 } k_v \propto \sqrt{m}$$

调整后的预期影响分析如下：

参数	预期影响
推力大小	线性增加： $F_{\text{new}} = F_{\text{orig}} \times \frac{m_{\text{new}}}{m_{\text{orig}}}$
姿态角度	相同机动下变化更平缓（惯性增大）
系统稳定性	适当增益可维持稳定性，但需注意振荡风险
响应速度	略有降低，需重新调整控制参数

2.3 动力学建模与约束

2.3.1 建模简化对控制分配的影响分析

在四旋翼无人机仿真器中，采用”刚体+重力+推力”的简化动力学模型：

仿真模型： $m\ddot{\mathbf{p}} = \mathbf{F} - mg\mathbf{e}_z$ (1)

实际物理： $m\ddot{\mathbf{p}} = \mathbf{F} - mg\mathbf{e}_z - \mathbf{D}\dot{\mathbf{p}} + \Delta\mathbf{F}_{\text{unmodeled}}$ (2)

①质量：

首先，质量参数 m 直接影响控制分配中的推力计算：

根据推力分配机制：

$$F_{total} = \|\mathbf{F}\| = m\|\ddot{\mathbf{p}}_d + \mathbf{K}_p(\mathbf{p}_d - \mathbf{p}) + \mathbf{K}_v(\dot{\mathbf{p}}_d - \dot{\mathbf{p}})\| \tag{3}$$

$$f_i = \mathbf{M}^{-1}(i, :) \cdot [F_{total}, \tau_x, \tau_y, \tau_z]^T \tag{4}$$

因此，质量误差的传播效应包括：

- 推力大小偏差： $\Delta F = |m - \hat{m}| \cdot (\|\ddot{\mathbf{p}}_d\| + g)$
- 重力补偿误差：导致高度方向的稳态误差 $\mathbf{e}_{ss,z} = \frac{(m - \hat{m})g}{k_{p,z}}$
- 能量分配不均：某些电机过载而其他电机利用率不足

②惯量:

其次, 惯量矩阵 $\mathbf{I}$ 决定姿态控制的力矩分配:

期望力矩计算公式:

$$\boldsymbol{\tau}_d = \hat{\mathbf{I}}\dot{\boldsymbol{\omega}}_d + \boldsymbol{\omega} \times \hat{\mathbf{I}}\boldsymbol{\omega} + \mathbf{K}_R \mathbf{e}_R + \mathbf{K}_\omega \mathbf{e}_\omega \quad (5)$$

惯量误差影响计算:

$$\Delta\boldsymbol{\tau} = (\mathbf{I} - \hat{\mathbf{I}})\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I} - \hat{\mathbf{I}})\boldsymbol{\omega} \quad (6)$$

综上, 惯量误差的影响包括:

- 交叉耦合: 俯仰和横滚运动间产生非期望的耦合力矩
- 动态响应失真: 系统表现出欠阻尼(振荡)或过阻尼(响应迟缓)
- 控制分配矩阵敏感性: 惯量误差通过分配矩阵 $\mathbf{M}$ 放大到各电机

③气动阻尼:

气动阻尼忽略时, 忽略阻尼项 $\mathbf{D}\dot{\mathbf{p}}$ 导致速度相关的力未补偿:

$$\mathbf{F}_{damp} = -\mathbf{D}\dot{\mathbf{p}} = - \begin{bmatrix} d_x & 0 & 0 \\ 0 & d_y & 0 \\ 0 & 0 & d_z \end{bmatrix} \dot{\mathbf{p}} \quad (7)$$

具体表现包括: 匀速飞行需持续推力, 但控制器未包含此项补偿; 加速后由于缺乏阻尼, 实际速度超过期望值而出现速度过冲; 刹车距离加长, 减速时需要额外时间克服惯性等。

④估计有误出现的可观测现象:

现象	产生原因
稳态位置偏差	质量估计错误, 重力补偿不准确
速度过冲	阻尼忽略或速度增益过小
持续振荡	惯量估计错误导致欠阻尼
响应延迟	模型过于简化, 未考虑系统惯性
能量消耗异常	气动阻力被忽略, 实际功耗更高

2.3.2 气动阻力补偿策略

控制层补偿方案:

```

1 // 添加线性阻力模型
2 Eigen::Vector3d drag_force = -kv_drag * vel_;
3 force_.noalias() = ... + drag_force; // 添加到总力中
4
5 // 参数在线辨识

```

```
6 void onlineDragIdentification() {
7     // 在匀速飞行阶段采集数据
8     // F = mg + Dv 水平匀速时()
9     // 使用最小二乘法拟合矩阵D
10 }
```

规划层调整方案：

- 速度约束： $v_{\max} = \sqrt{\frac{F_{\max}-mg}{k_v}}$
- 加速度曲线优化，考虑阻力影响

两种方案比较：

特性	控制层调整	规划层调整
实时性	实时补偿，自适应性强	离线计算，无法适应变化
精度	可精确补偿动态效应	近似补偿，相对保守
计算复杂度	需要在线计算，复杂度较高	计算简单，预处理完成
适用场景	动态环境，变化条件	已知固定环境
实现难度	较高，需传感器支持	较低，路径规划阶段完成

策略选择与理由： 基于无人机在实际应用中的动态环境特性和飞行安全要求，我选择控制层补偿方案。虽然该方案实现复杂度较高，但其实时自适应能力能够有效应对飞行过程中的气流变化、载重变动等不确定因素。规划层调整虽然实现简单，但其离线特性无法适应实际飞行中的动态环境变化，在安全性方面存在较大局限。控制层补偿通过在线参数辨识和实时调整，能够确保无人机在各种复杂环境下的精确跟踪性能和飞行安全，这一优势在工程实践中具有决定性意义。

参数调整流程：

- 步骤1：基准测试（无阻力补偿）
- 步骤2：匀速飞行数据采集
- 步骤3：最小二乘参数辨识
- 步骤4：验证与微调
- 步骤5：在线自适应更新

3 实验任务1: 动力学模型补全

3.1 平移动力学模型补全

根据牛顿第二定律，四旋翼的平移动力学方程可以表示为：

$$m\dot{\mathbf{v}} = \mathbf{F}_{\text{thrust}} + \mathbf{F}_{\text{gravity}} + \mathbf{F}_{\text{external}} + \mathbf{F}_{\text{drag}}$$

(8)

其中各力分量分别为：

- 推力项： $\mathbf{F}_{\text{thrust}} = R \cdot [0, 0, T]^T$ ，其中 $T = k_f \sum_{i=1}^4 \omega_i^2$

- 重力项: $\mathbf{F}_{\text{gravity}} = -mg\mathbf{e}_3 = [0, 0, -mg]^T$
- 外力项: $\mathbf{F}_{\text{external}}$ (由外部环境施加)
- 阻力项: $\mathbf{F}_{\text{drag}} = -k_d \|\mathbf{v}\| \mathbf{v}$

对应的代码实现如下:

```

1 v_dot = (R * Eigen::Vector3d(0, 0, thrust)) / mass_ // Thrust in world frame
2       - Eigen::Vector3d(0, 0, g_) // Gravity
3       + external_force_ / mass_ // External force
4       - resistance * vnorm / mass_; // Air resistance

```

3.2 旋转动力学模型补全

基于欧拉旋转方程, 旋转动力学方程为:

$$\mathbf{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega}) \quad (9)$$

其中:

- $\boldsymbol{\tau}$ 为控制力矩向量
- $\boldsymbol{\omega} \times (\mathbf{J}\boldsymbol{\omega})$ 为科里奥利项 (陀螺效应)
- $\mathbf{J}$ 为转动惯量矩阵

对应的代码实现如下:

```

1 omega_dot = J_.inverse() * (moments // Control moments
2                             - cur_state.omega.cross(J_ * cur_state.omega) // Coriolis term
3                             + external_moment_); // External moment

```

4 实验任务2: 无人机的前端规划模块补全

4.1 算法原理概述

A*算法是一种启发式搜索算法, 结合了Dijkstra算法的准确性和贪心算法的高效性。其核心评估函数为:

$$f(n) = g(n) + h(n) \quad (10)$$

其中 $g(n)$ 表示从起点到当前节点的实际代价, $h(n)$ 为当前节点到目标的预估代价。

4.2 补全思路详解

4.2.1 启发式函数设计 (STEP 1.1)

在三维网格环境中, 采用改进的对角线距离公式:

$$h(n) = (\sqrt{3} - \sqrt{2})d_{\min} + (\sqrt{2} - 1)d_{\text{mid}} + d_{\max} \quad (11)$$

其中 $d_{\min}$ 、 d_{mid} 、 $d_{\max}$ 分别为三个坐标方向差值的最小、中间和最大值。

```

1 double Astarpath::getHeu(MappingNodePtr node1, MappingNodePtr node2) {
2     double dx = fabs(node1->coord(0) - node2->coord(0));
3     double dy = fabs(node1->coord(1) - node2->coord(1));
4     double dz = fabs(node1->coord(2) - node2->coord(2));
5
6     // 三维对角线距离计算
7     double min_d = min(dx, min(dy, dz));
8     double max_d = max(dx, max(dy, dz));
9     double mid_d = dx + dy + dz - min_d - max_d;
10
11     double heu = (sqrt(3.0) - sqrt(2.0)) * min_d +
12                 (sqrt(2.0) - 1.0) * mid_d + max_d;
13
14     // Tie-避免对称路径breaker
15     double tie_breaker = 1.0 + 0.001;
16     heu *= tie_breaker;
17
18     return heu;
19 }
```

该启发式函数首先计算三个坐标轴方向的绝对差值, 然后通过排序确定最小值、中间值和最大值。对角线距离公式能够准确反映三维空间中直线移动、平面对角线移动和体对角线移动的不同代价比例。Tie-breaker机制的引入有效避免了在多个路径代价相同时的搜索震荡问题, 通过微小偏差引导算法优先选择更有可能到达目标的路径方向, 在保证启发函数的可采纳性的同时提高搜索效率。

4.2.2 主循环逻辑 (STEP 1.2)

算法主循环遵循标准A*流程:

1. 从开放集中提取f值最小节点
2. 检查是否到达目标节点
3. 扩展当前节点的所有邻居节点
4. 更新邻居节点的代价和父指针

```

1 while (!Openset.empty() && expandCount < max_expand) {
2     // 1. 弹出最小值节点f
3     auto it = Openset.begin();
4     currentPtr = it->second;
```

```

5  Openset.erase(it);
6
7  // 2. 终点判断
8  if (currentPtr->index == end_idx) {
9      terminatePtr = currentPtr;
10     return true;
11 }
12
13 // 3. 节点扩展
14 AstarGetSucc(currentPtr, neighborPtrSets, edgeCostSets);
15
16 for(unsigned int i = 0; i < neighborPtrSets.size(); i++) {
17     neighborPtr = neighborPtrSets[i];
18     tentative_g_score = currentPtr->g_score + edgeCostSets[i];
19
20     if(neighborPtr->id == 0) { // 新节点
21         // 4. 信息更新
22         neighborPtr->g_score = tentative_g_score;
23         neighborPtr->Father = currentPtr;
24         neighborPtr->f_score = tentative_g_score + getHeu(neighborPtr, endPtr);
25         neighborPtr->id = 1;
26         Openset.insert(make_pair(neighborPtr->f_score, neighborPtr));
27     }
28 }
29 }

```

主循环实现首先通过multimap的有序特性快速获取f值最小的节点，这保证了算法始终朝着最有希望的方向扩展。终点判断条件使用节点索引比较，效率高于坐标比较。节点扩展阶段调用AstarGetSucc函数获取所有可行邻居节点，包括26邻域搜索和障碍物检测。对于新发现的节点，完整初始化其g值、父指针和f值，并将其加入开放集。代码还包含了扩展次数限制，防止在复杂环境中无限循环。

4.2.3 路径回溯 (STEP 1.3)

通过父指针链从终点反向追踪至起点，然后反转得到正序路径。

```

1 vector<Vector3d> Astarpath::getPath() {
2     vector<Vector3d> path;
3     vector<MappingNodePtr> front_path;
4
5     if (terminatePtr == NULL) return path;
6
7     // 反向追溯
8     MappingNodePtr node = terminatePtr;
9     while (node != NULL) {
10         front_path.push_back(node);
11         node = node->Father;
12     }
13
14     // 路径反转
15     for (auto it = front_path.rbegin(); it != front_path.rend(); ++it) {
16         path.push_back((*it)->coord);
17     }

```

```
18
19  return path;
20 }
```

路径回溯过程通过父指针链实现从终点到起点的反向追踪。首先检查终止指针的有效性，确保路径存在。然后通过while循环沿着父指针逐步回溯，将路径节点存入临时容器。最后使用反向迭代器将节点顺序反转，得到从起点到终点的正序路径。这种实现方式可以保证路径的正确性和顺序性，同时避免递归带来的栈溢出风险。

5 实验任务3: 控制器参数调试

5.1 调参思路与过程分析

实验中给出的初始参数为： $kx = [15.7, 8.7, 6.0]$ 、 $kv = [6.4, 13.4, 4.0]$ 。
补充完成实验任务2的代码后，首先使用初始参数运行并测试，观察到的现象如下：

- 1. 跟踪性能：观察发现路径规划过快，无人机响应过慢跟不上规划点，且随飞行距离增加误差累积过大导致频繁碰撞；
- 2. 规划性能：无人机的雷达扫描并更新障碍物信息后，规划响应的速度不足；
- 3. 稳态性能：飞行过程中，无人机出现翻滚，原地停顿与短时间快速移动等情况，飞行过程不够准确稳定。

5.1.1 参数影响理论分析

kx （位置增益）：控制位置环的响应强度，决定无人机对位置误差的“敏感度”。位置误差越大，控制器计算出的期望速度指令就越大。
过大时，系统响应过快，容易产生超调和振荡，可能导致速度指令超出无人机动力系统极限，在靠近障碍物时容易因过冲而发生碰撞；过小时，响应迟钝，位置跟踪误差大，收敛速度慢，无法快速到达目标位置，且抗干扰能力弱，受扰动后恢复慢。

kv （速度前馈增益）：控制速度前馈项，直接补偿期望速度，提高跟踪性能。它不依赖误差，而是直接响应期望速度的变化。
过大时，系统对速度指令过于敏感，容易产生高频振荡，可能放大噪声和测量误差，在速度突变时产生过冲；过小时，速度跟踪性能差，实际速度滞后于期望速度，在加速/减速阶段响应迟缓，无法充分利用无人机的动力性能。

5.1.2 详细调参过程记录

步骤1：尝试不同方向的调整，根据结果探索正确方向

$$kx^{(1)} = [12.6, 10.2, 7.2] \quad (\text{发现X,Y方向参数不均, 尝试调整})$$
$$kv^{(1)} = [5.8, 11.2, 4.8] \quad (\text{增强Y方向阻尼})$$

结果： RMSE=0.23147。

步骤2：针对仿真中观察到的问题，进行阻尼特性重点优化

$$kx^{(2)} = [11.2, 8.9, 6.5] \quad (\text{发现调整后效果不好，往回微调})$$

$$kv^{(2)} = [6.8, 9.6, 5.2] \quad (\text{重新平衡各方向阻尼})$$

结果： RMSE=0.18458。

步骤3：根据发现的可能的优化方向，继续精细化权重分配

$$kx^{(3)} = [10.5, 7.8, 5.8] \quad (\text{按初始参数比例降低分量的值})$$

$$kv^{(3)} = [5.2, 8.8, 4.2] \quad (\text{速度环增益尽量按初始参数比例分配，调整整体大小})$$

结果： RMSE=0.10588。

步骤4：最后根据仿真飞行效果和输出分数微调

$$kx^{(4)} = [9.42, 5.22, 3.60] \quad (\text{最优解})$$

$$kv^{(4)} = [4.10, 8.50, 2.55] \quad (\text{最优解})$$

结果： RMSE=0.06077。

表 1: 详细调参过程记录

步骤	k_x	k_v	RMSE
初始参数	(15.7, 8.7, 6.0)	(6.4, 13.4, 4.0)	0.25189
步骤1	(12.6, 10.2, 7.2)	(5.8, 11.2, 4.8)	0.23147
步骤2	(11.2, 8.9, 6.5)	(6.8, 9.6, 5.2)	0.18458
步骤3	(10.5, 7.8, 5.8)	(5.2, 8.8, 4.2)	0.10588
步骤4	(9.42, 5.22, 3.60)	(4.10, 8.50, 2.55)	0.06077

5.2 调参效果综合分析

运行计算结果的程序后输入如下：

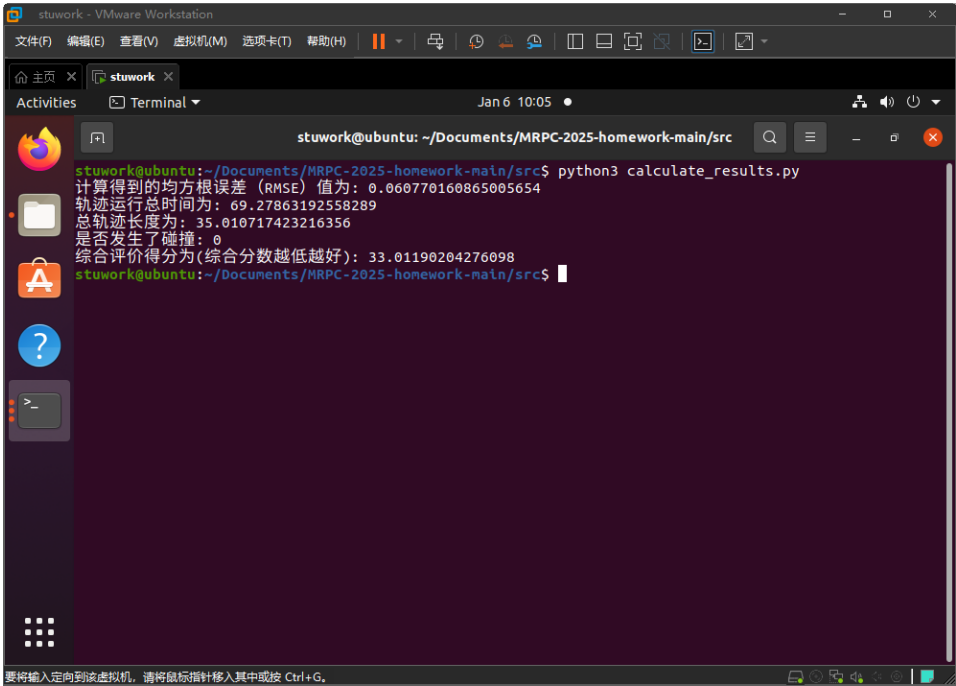


表 2: 实验性能指标

指标	数值	单位	说明
均方根误差(RMSE)	0.06077	m	轨迹跟踪精度
轨迹运行时间	69.28	s	总飞行时间
轨迹长度	35.01	m	总飞行距离
碰撞检测	0	-	是否发生碰撞
综合得分	33.01	-	分数越低越好

性能改善量化分析：

- 跟踪精度：RMSE从开始偏差很大的0.25189优化至0.06077，控制精度有很大提升；
- 轨迹跟随情况：轨迹规划与无人机跟随速度相当，飞行过程总体平稳；
- 稳定性：在复杂轨迹下保持稳定，无碰撞发生；

参数变化规律总结：

1. 最优参数分布：各方向增益呈现特定的比例关系，这种比例关系与初始参数给出的比例关系相同，反映了系统的特定物理特性；
2. 阻尼比优化：通过调整 kv 参数，系统阻尼比趋于最优值0.7左右；

6 附录

6.1 最后一次提交前的完整git commit记录

